Dataset Description “winequality-red”

The publicly available Wine Quality dataset, published in the UCI Machine Learning Repository, contains physicochemical measurements and sensory quality ratings of Portuguese “Vinho Verde” wines from the Minho region in north-west Portugal. The data was collected between 2004 and 2007 and consists of two separate datasets, one for red and one for white wines. The attached dataset is the **red wine** dataset, which contains 1,599 wine samples.

The dataset was originally presented by Cortez et al. (2009) in the study *“Modeling wine preferences by data mining from physicochemical properties”*, which explores whether the quality of a wine, as judged by human experts, can be predicted from objective laboratory tests. Wine quality is usually assessed through sensory tasting, which is subjective, time-consuming and expensive. Identifying which chemical properties influence quality could therefore support wine producers in quality control, production decisions and pricing.

The procedure was as follows. Each wine sample was first analyzed in a laboratory, where 11 physicochemical properties were measured, such as acidity, sugar, Sulphur dioxide and alcohol content. Afterwards, each wine was evaluated in blind tastings by at least three wine experts, who rated it on a scale from 0 (very bad) to 10 (excellent). The final quality score is the median of the experts’ ratings.

Dataset composition:

- 1,599 observations, each one representing a single red wine sample.
- 11 numerical input features obtained from physicochemical tests:
    - fixed acidity, volatile acidity and citric acid
    - residual sugar
    - chlorides
    - free sulfur dioxide and total sulfur dioxide
    - density
    - pH
    - sulphates
    - alcohol
- 1 target variable, **quality**, a score based on sensory data. Although the scale ranges from 0 to 10, only the values 3 to 8 appear in the dataset, and most wines are rated 5 or 6. For this project the scores are grouped into three classes:
    - Low quality (3–4) = 0
    - Medium quality (5–6) = 1
    - High quality (7–8) = 2

The aim of this project is to apply data mining and machine learning techniques to classify red wines into these quality classes based on their physicochemical properties, and to identify which properties have the greatest influence on wine quality.

1. Data Mining and Dataset Exploration

1.1 Importing libraries

This segment involves importing essential libraries for data analysis and machine learning. These include pandas for data manipulation, matplotlib and seaborn for data visualization.

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
```

[86] ✓ 0.0s Python

1.2 Importing the Dataset

Importing the dataset using pandas

```
df = pd.read_csv("winequality-red.csv")

df
```

[132] ✓ 0.0s Python

1.3 Display first few rows of the dataset

The .head() function is used to display the first few rows of the loaded dataset. The structure and content of the data consist of 20 numerical features including the target.

```
df.head()
```

[135] ✓ 0.0s Python

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	quality
0	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5
1	7.8	0.88	0.00	2.6	0.098	25.0	67.0	0.9968	3.20	0.68	9.8	5
2	7.8	0.76	0.04	2.3	0.092	15.0	54.0	0.9970	3.26	0.65	9.8	5
3	11.2	0.28	0.56	1.9	0.075	17.0	60.0	0.9980	3.16	0.58	9.8	6
4	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	quality
0	7.4	0.700	0.00	1.9	0.076	11.0	34.0	0.99780	3.51	0.56	9.4	5
1	7.8	0.880	0.00	2.6	0.098	25.0	67.0	0.99680	3.20	0.68	9.8	5
2	7.8	0.760	0.04	2.3	0.092	15.0	54.0	0.99700	3.26	0.65	9.8	5
3	11.2	0.280	0.56	1.9	0.075	17.0	60.0	0.99800	3.16	0.58	9.8	6
4	7.4	0.700	0.00	1.9	0.076	11.0	34.0	0.99780	3.51	0.56	9.4	5
...	...	...	...	...	...	...	...	...	...	...	...	...
1594	6.2	0.600	0.08	2.0	0.090	32.0	44.0	0.99490	3.45	0.58	10.5	5
1595	5.9	0.550	0.10	2.2	0.062	39.0	51.0	0.99512	3.52	0.76	11.2	6
1596	6.3	0.510	0.13	2.3	0.076	29.0	40.0	0.99574	3.42	0.75	11.0	6
1597	5.9	0.645	0.12	2.0	0.075	32.0	44.0	0.99547	3.57	0.71	10.2	5
1598	6.0	0.310	0.47	3.6	0.067	18.0	42.0	0.99549	3.39	0.66	11.0	6

1599 rows × 12 columns

1.4 Displaying Info

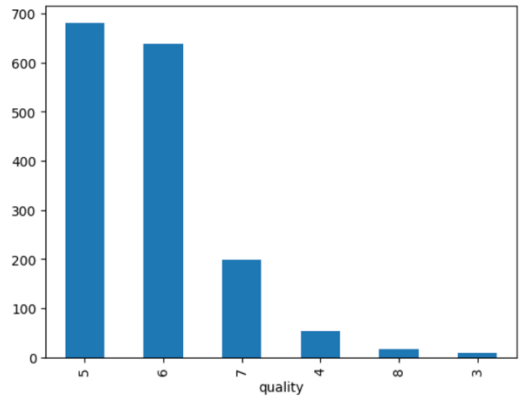
.info(): The function is used to demonstrate Index and Range, Column Names, Data Types, NonNull counts and Memory Usage which are 1599 entries, 0 to 1598 int and float data type.

```
df.info()
```
[88] ✓ 0.0s Python

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1599 entries, 0 to 1598
Data columns (total 12 columns):
 #   Column                Non-Null Count  Dtype  
---  ------                --------------  -----  
 0   fixed acidity         1599 non-null   float64
 1   volatile acidity      1599 non-null   float64
 2   citric acid           1599 non-null   float64
 3   residual sugar        1599 non-null   float64
 4   chlorides             1599 non-null   float64
 5   free sulfur dioxide   1599 non-null   float64
 6   total sulfur dioxide  1599 non-null   float64
 7   density               1599 non-null   float64
 8   pH                    1599 non-null   float64
 9   sulphates             1599 non-null   float64
 10  alcohol               1599 non-null   float64
 11  quality               1599 non-null   int64  
dtypes: float64(11), int64(1)
memory usage: 150.0 KB
```

1.5 Counting values of each instance of Label (target)

```
df['quality'].value_counts().plot(kind='bar')
plt.show()
```
[89] ✓ 0.0s




```
df['quality'].value_counts()
```
[90] ✓ 0.0s

```
quality
5    681
6    638
7    199
4     53
8     18
3     10
Name: count, dtype: int64
```

1.6 Describing the statistical values of each feature

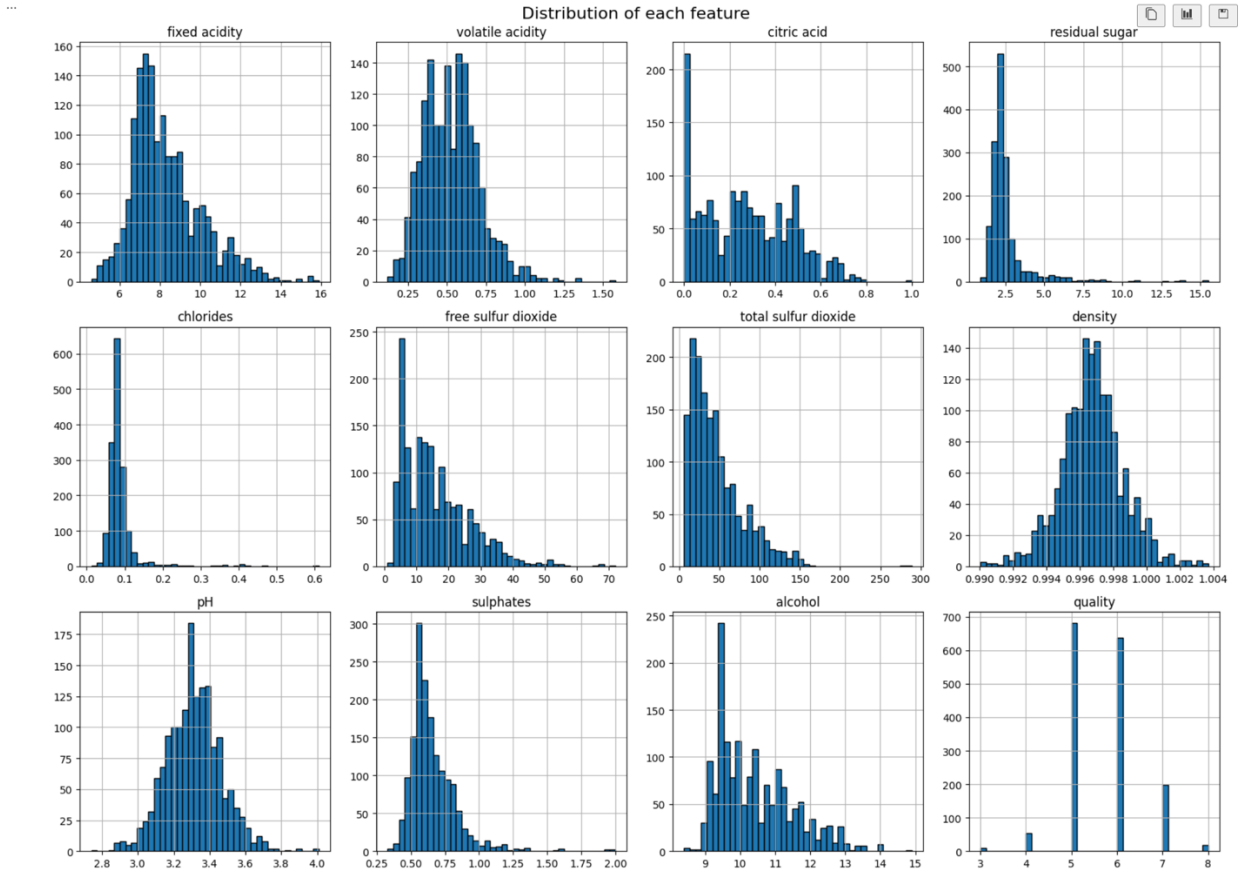
We get some information like mean, count, standard deviation, min and max.

```
df.describe()
```
[91] ✓ 0.0s Python

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	15
count	1599.000000	1599.000000	1599.000000	1599.000000	1599.000000	1599.000000	1599.000000	1599.000000	1599.000000	1599.000000	1599.000000	
mean	8.319637	0.527821	0.270976	2.538806	0.087467	15.874922	46.467792	0.996747	3.311113	0.658149	10.422983	
std	1.741096	0.179060	0.194801	1.409928	0.047065	10.460157	32.895324	0.001887	0.154386	0.169507	1.065668	
min	4.600000	0.120000	0.000000	0.900000	0.012000	1.000000	6.000000	0.990070	2.740000	0.330000	8.400000	
25%	7.100000	0.390000	0.090000	1.900000	0.070000	7.000000	22.000000	0.995600	3.210000	0.550000	9.500000	
50%	7.900000	0.520000	0.260000	2.200000	0.079000	14.000000	38.000000	0.996750	3.310000	0.620000	10.200000	
75%	9.200000	0.640000	0.420000	2.600000	0.090000	21.000000	62.000000	0.997835	3.400000	0.730000	11.100000	
max	15.900000	1.580000	1.000000	15.500000	0.611000	72.000000	289.000000	1.003690	4.010000	2.000000	14.900000	

1.7 Bins to show the distribution of each feature

Here we see that that some features are skewed, this skewness affects a model's performance, so it must be treated to make the distribution normal so the model will learn without any bias and perform good.

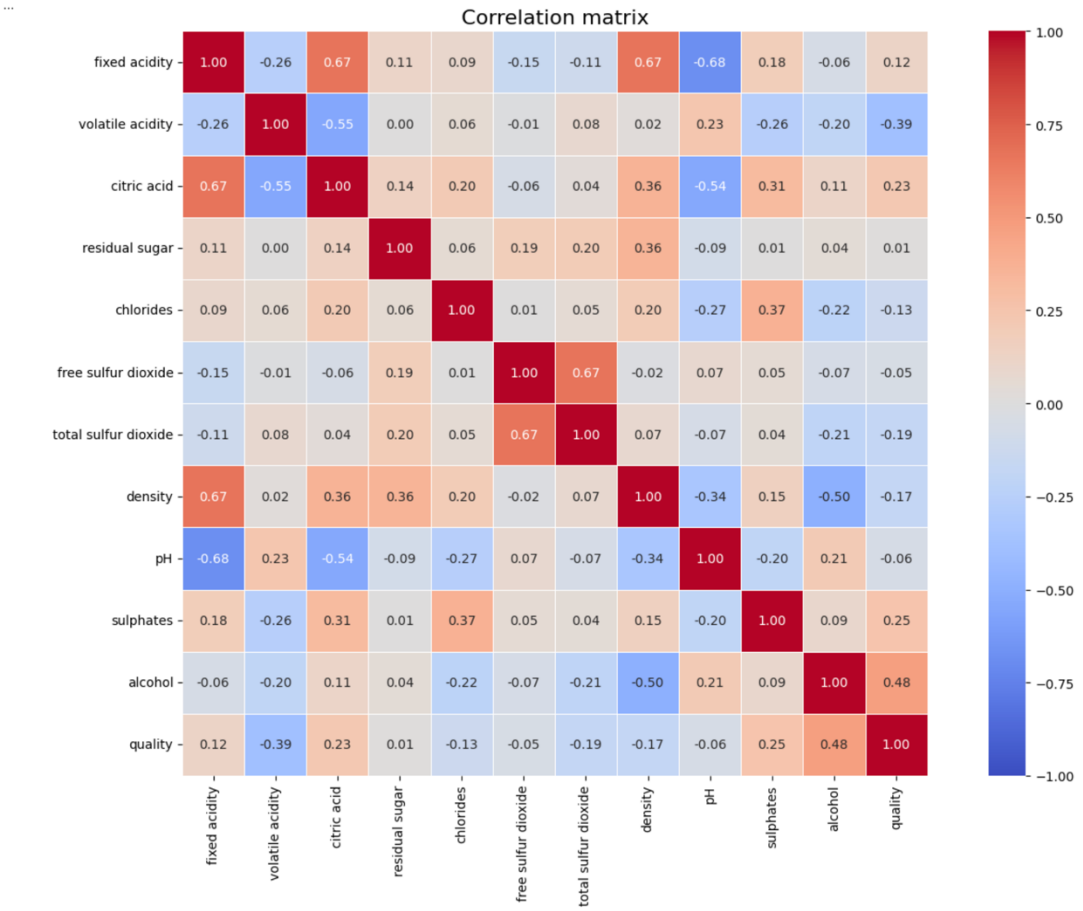


1.8 Correlations

The correlation matrix shows how strongly each feature is related to every other feature, and to the target (quality). Correlation values range from **−1 to 1**:

- **0** means there is no correlation between two features.
- **1** means they are perfectly positively correlated, so they are directly proportional: when one feature increases, the other also increases.
- **−1** means they are perfectly negatively correlated, so they are inversely proportional: when one feature increases, the other decreases.

A strong correlation between input features (multicollinearity) is not recommended, because the features carry the same information. Such features should either be dropped or combined using statistical techniques such as PCA. A correlation between a feature and the target, on the other hand, is useful, because it shows that the feature can help predict quality.



This heatmap shows all the correlation values. The following groups of features are strongly correlated with each other:

- (fixed acidity, citric acid, density, pH)
  - fixed acidity – pH: −0.68
  - fixed acidity – citric acid: 0.67
  - fixed acidity – density: 0.67
  - citric acid – pH: −0.54
- (free sulfur dioxide, total sulfur dioxide): 0.67
- volatile acidity – citric acid: −0.55, and density – alcohol: −0.50

The relationship of the features with the target, **quality**, is:

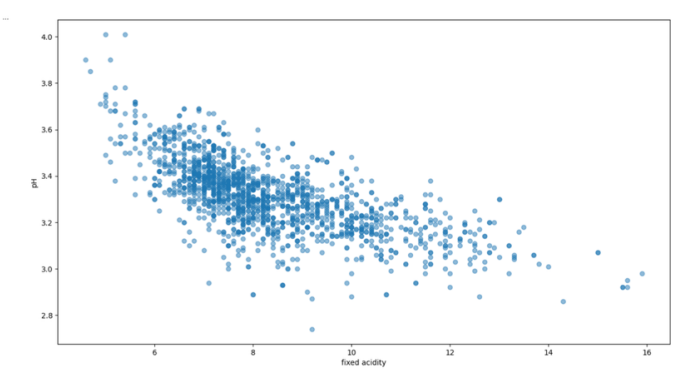
- **alcohol** has the strongest positive correlation (0.48): wines with more alcohol tend to receive higher quality scores.

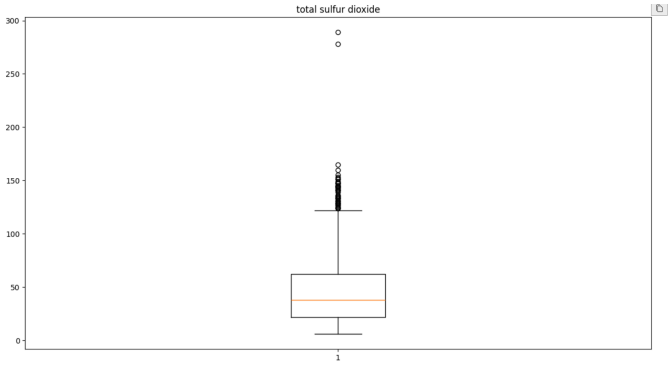
- **volatile acidity** has the strongest negative correlation (−0.39): too much of it gives the wine a vinegar-like taste and lowers its quality.
- **sulphates** (0.25) and **citric acid** (0.23) have a weak positive correlation.
- **residual sugar**, **free sulfur dioxide** and **pH** have almost no correlation with quality (close to 0).

No feature has a very strong correlation with quality (all are below 0.5), which shows that quality depends on a combination of several features. The groups of correlated features will be treated in section 3 using PCA.

1.9 Functions to plot

I wrote two functions to help visualize the data. The first draws a **scatter plot** to show the relationship between two features. The second draws a **box plot**, which shows the quartiles of a feature and helps to detect outliers.





2. Data Preprocessing

2.1 Checking for null and duplicate values

If there are missing values, we must handle them with the best technique, which depends on the nature of the dataset. Duplicate entries are checked as well. Below I used pandas functions to check for null values and found no null values at all.

```
df.isna().sum()
```
[100] ✓ 0.0s

```
fixed acidity           0
volatile acidity        0
citric acid             0
residual sugar          0
chlorides               0
free sulfur dioxide     0
total sulfur dioxide    0
density                 0
pH                      0
sulphates               0
alcohol                 0
quality                 0
dtype: int64
```

```
df.duplicated().sum()
```
[101] ✓ 0.0s

```
np.int64(240)
```

The dataset contains **240 duplicate rows** (15% of the data). Duplicates give some wines more weight during training. They can also end up in both the training and the test set, which makes the model look better than it really is. Since duplicate rows add no new information, I removed them:

```
print('Duplicate rows:', df.duplicated().sum())
```
[102] ✓ 0.0s

```
Duplicate rows: 240
```

```
df = df.drop_duplicates().reset_index(drop=True)
print('Shape after removing duplicates:', df.shape)

df_clean = df.copy()
```
[103] ✓ 0.0s

```
Shape after removing duplicates: (1359, 12)
```

2.2 Normalisation

Normalisation will be applied to the features that are skewed. For normalisation I chose a **logarithmic transformation**, which works well for features with skewed behaviour. It uses a logarithmic function to make the distribution closer to normal. Because it compresses large values more than small ones, it also reduces the effect that outliers can have on the model.

I chose the features with skewness above 0.5 for the transformation. All their values are greater than 0, so I used $\log(x)$ instead of $\log(x + 1)$.

```
df = df_clean.copy()

df['chlorides'] = np.log(df['chlorides'])
df['residual sugar'] = np.log(df['residual sugar'])
df['sulphates'] = np.log(df['sulphates'])
df['total sulfur dioxide'] = np.log(df['total sulfur dioxide'])
df['free sulfur dioxide'] = np.log(df['free sulfur dioxide'])
df['fixed acidity'] = np.log(df['fixed acidity'])
df['volatile acidity'] = np.log(df['volatile acidity'])
df['alcohol'] = np.log(df['alcohol'])

df.skew()   # check the result
```

[106] ✓ 0.0s

```
fixed acidity           0.348804
volatile acidity       -0.330795
citric acid             0.312726
residual sugar          1.765238
chlorides               1.887642
free sulfur dioxide    -0.220069
total sulfur dioxide   -0.078161
density                 0.044778
pH                      0.232032
sulphates               0.961460
alcohol                 0.663359
quality                 0.192407
dtype: float64
```

	Outliers	Skew before	Skew after
fixed acidity	11	0.35	0.30
volatile acidity	11	-0.33	-0.29
citric acid	1	0.31	0.31
residual sugar	96	1.77	0.59
chlorides	102	1.89	0.17
free sulfur dioxide	2	-0.22	-0.20
total sulfur dioxide	0	-0.08	-0.08
density	35	0.04	0.01
pH	28	0.23	0.07
sulphates	31	0.96	0.54
alcohol	1	0.66	0.65

Shape: (1359, 12)

The skewness of all the transformed features decreased. Chlorides and residual sugar are still skewed because of their extreme values, which are treated in the next step.

2.3 Treating outliers

Outliers affect a model's performance, because models are affected by very large or very small numbers. Outliers can give some features too much influence, which leads to a model with lower performance or a biased model, so treating them is essential.

For my dataset, I used the quartiles (the IQR method) to detect outliers. To treat them I used **clipping**, which moves each outlier to the lower or upper bound. I chose this because 251 rows (18.5% of the dataset) contain at least one outlier. Removing them would lose too much data, including many of the rare low- and high-quality wines. Clipping lets me keep all the records while making sure the outliers do not affect the model in an unexpected way.

```python
def clip_outliers(column):
    data = df[column]
    q1 = np.percentile(data, 25)
    q3 = np.percentile(data, 75)
    iqr = q3 - q1
    lower_bound = q1 - 1.5 * iqr
    upper_bound = q3 + 1.5 * iqr
    return np.clip(data, lower_bound, upper_bound)
```

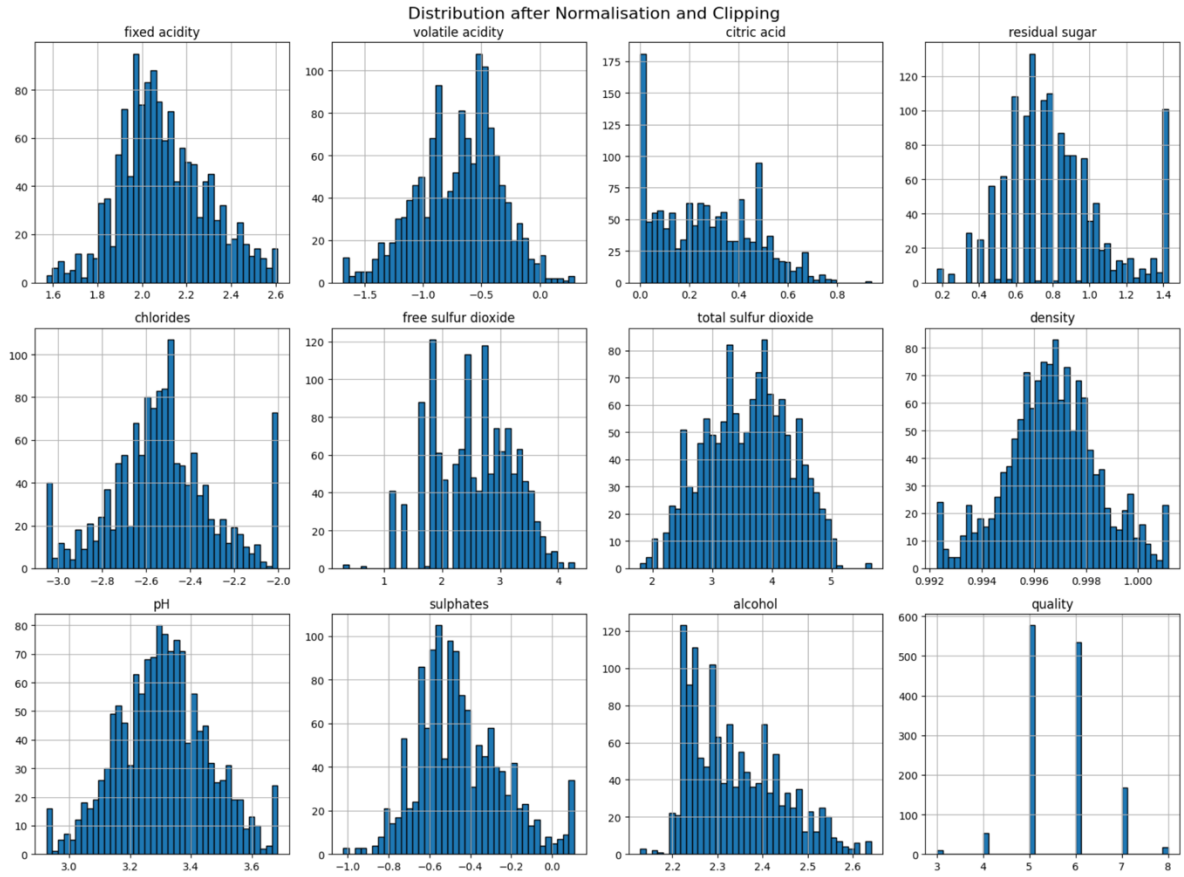
```python
features = df.columns.drop('quality')   # never clip the target

# Count outliers and skewness BEFORE clipping
outliers_before = {}
skew_before = df[features].skew()
for col in features:
    q1, q3 = df[col].quantile([0.25, 0.75])
    iqr = q3 - q1
    outliers_before[col] = ((df[col] < q1 - 1.5 * iqr) | (df[col] > q3 + 1.5 * iqr)).sum()

# Apply clipping
df['fixed acidity'] = clip_outliers('fixed acidity')
df['volatile acidity'] = clip_outliers('volatile acidity')
df['citric acid'] = clip_outliers('citric acid')
df['residual sugar'] = clip_outliers('residual sugar')
df['chlorides'] = clip_outliers('chlorides')
df['free sulfur dioxide'] = clip_outliers('free sulfur dioxide')
df['total sulfur dioxide'] = clip_outliers('total sulfur dioxide')
df['density'] = clip_outliers('density')
df['pH'] = clip_outliers('pH')
df['sulphates'] = clip_outliers('sulphates')
df['alcohol'] = clip_outliers('alcohol')

# Results
summary = pd.DataFrame({
    'Outliers': pd.Series(outliers_before),
    'Skew before': skew_before,
    'Skew after': df[features].skew()
}).round(2)
print(summary)
print('Shape:', df.shape)
```

2.4 Visualising distribution after normalisation and clipping



3. Feature Engineering and Standardisation

As I mentioned in section 1.8, some features are strongly correlated with each other. For these features I decided to use **PCA**, a dimensionality reduction technique that lets me create one new feature from several features.

The features in the groups (fixed acidity, citric acid, density, pH) and (free sulfur dioxide, total sulfur dioxide) will be treated, producing **pca_1** and **pca_2** respectively.

Before applying PCA, I standardised the features in each group. PCA is sensitive to scale, and in my data density has a standard deviation of only 0.0018, compared with about 0.2 for fixed acidity. Without scaling, PCA would almost ignore density.

```python
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

pca = PCA(n_components=1)
feature_1 = ['fixed acidity', 'citric acid', 'density', 'pH']
feature_2 = ['free sulfur dioxide', 'total sulfur dioxide']

# standardise each group before PCA so every feature counts equally
pca_1 = pca.fit_transform(StandardScaler().fit_transform(df[feature_1]))
print('pca_1 explained variance:', pca.explained_variance_ratio_)

pca_2 = pca.fit_transform(StandardScaler().fit_transform(df[feature_2]))
print('pca_2 explained variance:', pca.explained_variance_ratio_)

df['pca_1'] = pca_1
df['pca_2'] = pca_2
df.info()
```

[115] ✓ 0.0s Python

```
pca_1 explained variance: [0.67047126]
pca_2 explained variance: [0.89436845]
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1359 entries, 0 to 1358
Data columns (total 15 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   fixed acidity         1359 non-null   float64
 1   volatile acidity      1359 non-null   float64
 2   citric acid           1359 non-null   float64
 3   residual sugar        1359 non-null   float64
 4   chlorides             1359 non-null   float64
 5   free sulfur dioxide   1359 non-null   float64
 6   total sulfur dioxide  1359 non-null   float64
 7   density               1359 non-null   float64
 8   pH                    1359 non-null   float64
 9   sulphates             1359 non-null   float64
 10  alcohol               1359 non-null   float64
 11  quality               1359 non-null   int64
 12  quality_class         1359 non-null   int64
 13  pca_1                 1359 non-null   float64
 14  pca_2                 1359 non-null   float64
dtypes: float64(13), int64(2)
memory usage: 159.4 KB
```

The explained variance shows how much information each new feature keeps:

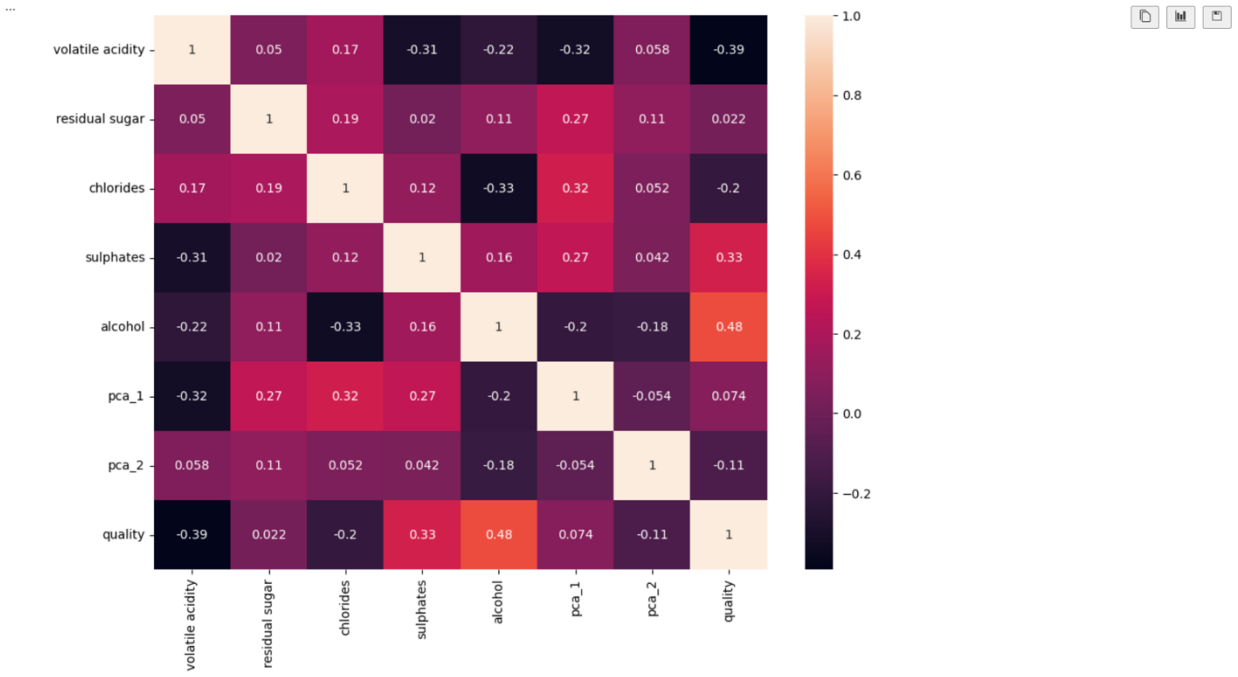
- **pca_1** keeps **67.0%** of the information of the four acidity features.
- **pca_2** keeps **89.4%** of the information of the two sulfur features.

I have now added the new features in place of the other features, so I need to remove the features I used in PCA. The output above shows all the features before the unnecessary ones are removed.

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1359 entries, 0 to 1358
Data columns (total 8 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   volatile acidity  1359 non-null   float64
 1   residual sugar    1359 non-null   float64
 2   chlorides         1359 non-null   float64
 3   sulphates         1359 non-null   float64
 4   alcohol           1359 non-null   float64
 5   pca_1             1359 non-null   float64
 6   pca_2             1359 non-null   float64
 7   quality           1359 non-null   int64
dtypes: float64(7), int64(1)
memory usage: 85.1 KB
```

After applying PCA and removing the original features, the dataset has **8 columns**: 7 features and the target (quality).

3.1 View correlations after normalisation, PCA and treating outliers



The multicollinear features have been treated: we no longer see features with high correlation, as we had before applying PCA and normalisation. The strongest correlation between two features is now only **0.33** (chlorides and alcohol), compared with up to 0.68 in section 1.8. The relationships with the target are kept:

- **alcohol:** 0.48
- **volatile acidity:** −0.39
- **sulphates:** 0.33

3.2 Standardisation

Standardisation, or Z-score normalisation, is a preprocessing technique that scales features to a mean of 0 and a standard deviation of 1. This ensures that all features contribute equally to a machine learning model and prevents those with larger scales from dominating the learning process. Unlike Min-Max scaling, standardisation does not restrict values to a specific range, and it is less affected by extreme values. For this reason, I used standardisation.

```
from sklearn.preprocessing import StandardScaler

# target: group quality into 3 classes (0 = low, 1 = medium, 2 = high)
y = pd.cut(df['quality'], bins=[2, 4, 6, 8], labels=[0, 1, 2]).astype(int)
X = df.drop('quality', axis=1)

scaler = StandardScaler()
X = scaler.fit_transform(X)

print(y.value_counts().sort_index())
```

[118] ✓ 0.0s Python

```
quality
0      63
1    1112
2     184
Name: count, dtype: int64
```

Together with standardisation, I separated the target (quality) from the rest of the features. Some quality scores have very few samples (only 10 wines are rated 3 and 17 are rated 8), so I grouped them into three classes:

- **0 = Low** (3–4): 63 wines
- **1 = Medium** (5–6): 1,112 wines
- **2 = High** (7–8): 184 wines

3.3 Splitting the dataset into train and test sets

I split the dataset into training and test sets using a function from sklearn, with 80% for training and 20% for testing. The training set is used to train the models, and the test set is used to evaluate their performance. Because the classes are imbalanced, I used stratify=y, so that both sets keep the same proportion of low, medium and high wines.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

print('Train:', X_train.shape, ' Test:', X_test.shape)
print('Train classes:', y_train.value_counts().sort_index().tolist())
print('Test classes: ', y_test.value_counts().sort_index().tolist())
```

[119] ✓ 0.0s

```
Train: (1087, 7)  Test: (272, 7)
Train classes: [50, 890, 147]
Test classes:  [13, 222, 37]
```

The training set contains **1,087 wines**: 50 low, 890 medium and 147 high.
The test set contains **272 wines**: 13 low, 222 medium and 37 high.

4. Models

For modelling, I chose three models, and I will also use them in cross-validation and hyperparameter tuning.

The chosen models are:

```
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, f1_score

def model_report(y_test, y_pred):
    print('Confusion Matrix:\n', confusion_matrix(y_test, y_pred))
    print('Classification Report:\n', classification_report(y_test, y_pred, zero_division=0))
    print('Accuracy:', round(accuracy_score(y_test, y_pred), 3))
    print('Macro F1-score:', round(f1_score(y_test, y_pred, average='macro'), 3))
```

[120] ✓ 0.3s Python

A. Random Forest Classifier

Random Forest Classifier is a machine learning algorithm that uses multiple decision trees to make predictions. Instead of relying on a single tree, it creates many trees and combines their results for better accuracy and stability. This helps reduce errors and prevents overfitting.

B. Gradient Boosting Classifier

Gradient Boosting Classifier is a machine learning algorithm that builds multiple weak models, usually decision trees, in a step-by-step process. Each new tree corrects the mistakes of the previous ones, making the model more accurate over time. Unlike Random Forest, which builds its trees independently, Gradient Boosting improves gradually by focusing on errors. It works well for complex patterns but can be sensitive to overfitting if not tuned properly.

C. Support Vector Machine

Support Vector Machine (SVM) is a machine learning algorithm used for classification. It works by finding the best boundary, or hyperplane, that separates the classes in the data, and it tries to maximise the margin between the classes. It is effective for high-dimensional data and works well with both linear and non-linear problems by using different kernel functions.

I also used different performance metrics to evaluate each model: accuracy, precision, recall, F1-score and the confusion matrix. Because the classes are imbalanced (82% of the wines are medium), I also report the **macro F1-score**, which gives equal importance to all three classes. A model that always predicts “medium” would already reach 81.6% accuracy on the test set, so accuracy alone can be misleading.

4.1 Training and evaluating the Gradient Boosting Classifier

I trained this model on the training set and then used the performance metrics to evaluate it.

Gradient Boosting

```
gb = GradientBoostingClassifier(random_state=42)
gb.fit(X_train, y_train)
gb_pred = gb.predict(X_test)
model_report(y_test, gb_pred)
```

[180] ✓ 0.3s

```
Confusion Matrix:
 [[  0  13   0]
 [  0 208  14]
 [  0  16  21]]
Classification Report:
              precision    recall  f1-score   support

           0       0.00      0.00      0.00        13
           1       0.88      0.94      0.91       222
           2       0.60      0.57      0.58        37

    accuracy                           0.84       272
   macro avg       0.49      0.50      0.50       272
weighted avg       0.80      0.84      0.82       272

Macro F1 score: 0.4965504720406681
Accuracy: 0.8419117647058824
```

4.1.1 Model performance analysis

A. Confusion matrix:

```
Confusion Matrix:
 [[  0  13   0]
 [  0 208  14]
 [  0  16  21]]
```

- **Class 0 (Low):** 0 correct predictions, 13 misclassified as 1 and 0 misclassified as 2.
- **Class 1 (Medium):** 208 correct predictions, 0 misclassified as 0 and 14 misclassified as 2.
- **Class 2 (High):** 21 correct predictions, 0 misclassified as 0 and 16 misclassified as 1.

B. Classification report:

- **Precision** shows the proportion of correct predictions among all the predictions made for a class.
  - Class 0: precision of 0 means the model never predicted a wine as low quality. It could not capture the pattern of this class.
  - Class 1: precision of 0.88 means 88% of the wines predicted as 1 are correct.
  - Class 2: precision of 0.60 means 60% of the wines predicted as 2 are correct.
- **Recall** measures how many wines of a class were correctly identified out of all the wines in that class.
  - Class 0: 0.00 means none of the low wines were correctly classified.
  - Class 1: 0.94 means 94% of the medium wines were correctly classified.
  - Class 2: 0.57 means 57% of the high wines were correctly classified.

- **F1-score** is the harmonic mean of precision and recall, used to balance the two.
    - Class 0: 0.00, which is the worst possible result.
    - Class 1: 0.91, which is very good at balancing precision and recall for this class.
    - Class 2: 0.58, which is moderate.

C. Accuracy:

The accuracy is 84.2%, but the macro F1-score is only 0.50. The accuracy is only 2.6% above the baseline of 81.6%, so it looks high mainly because most of the wines are medium.

To summarize:

The model's accuracy and performance metrics show that it is good at classifying class 1 and moderate for class 2, but it completely fails on class 0. There are only 50 low wines in the training set, so the model learned that predicting "medium" is almost always safe.

4.2 Training and evaluating the Support Vector Machine

I trained this model on the training set and then used the performance metrics to evaluate it.

Support Vector Machine

```
[181] ✓ 0.0s
from sklearn.svm import SVC
svm_model = SVC(kernel='rbf', C=1, random_state=42)  # RBF kernel to create a non-li
svm_model.fit(X_train, y_train)
y_pred = svm_model.predict(X_test)
```

```
[182] ✓ 0.0s
model_report(y_test, y_pred)
```

```
Confusion Matrix:
 [[  0  13   0]
 [  0 217   5]
 [  0  22  15]]
Classification Report:
              precision    recall  f1-score   support

           0       0.00      0.00      0.00        13
           1       0.86      0.98      0.92       222
           2       0.75      0.41      0.53        37

    accuracy                           0.85       272
   macro avg       0.54      0.46      0.48       272
weighted avg       0.80      0.85      0.82       272

Macro F1 score: 0.4806425346065586
Accuracy: 0.8529411764705882
```

4.2.1 Model performance analysis

A. Confusion matrix:

```
Confusion Matrix:
 [[  0  13   0]
 [  0 217   5]
 [  0  22  15]]
```

- **Class 0 (Low):** 0 correct predictions, 13 misclassified as 1 and 0 misclassified as 2.
- **Class 1 (Medium):** 217 correct predictions, 0 misclassified as 0 and 5 misclassified as 2.
- **Class 2 (High):** 15 correct predictions, 0 misclassified as 0 and 22 misclassified as 1.

B. Classification report:

- **Precision** shows the proportion of correct predictions among all the predictions made for a class.
  - Class 0: precision of 0 means the model never predicted low.
  - Class 1: precision of 0.86 means 86% of the wines predicted as 1 are correct.
  - Class 2: precision of 0.75 means 75% of the wines predicted as 2 are correct, the best precision for this class of the three models.
- **Recall** measures how many wines of a class were correctly identified out of all the wines in that class.
  - Class 0: 0.00 means none of the low wines were found.
  - Class 1: 0.98 means 98% of the medium wines were correctly classified.
  - Class 2: 0.41 means only 41% of the high wines were correctly classified.
- **F1-score** is the harmonic mean of precision and recall, used to balance the two.
  - Class 0: 0.00, which is the worst possible result.
  - Class 1: 0.92, which is very good.
  - Class 2: 0.53, which is considered poor to moderate.

C. Accuracy:

The accuracy is 85.3%, the highest of the three models, but the macro F1-score is only 0.48.

To summarize:

The model behaves much like Gradient Boosting, apart from accuracy. It has the highest accuracy mainly because it predicts class 1 for almost every wine. It is careful when predicting class 2 (high precision, low recall) and does not recognize class 0 at all.

4.3 Training and evaluating the Random Forest Classifier

I also trained this model on the training set and then used the performance metrics to evaluate it.

Random Forest Classifier

```
rfc = RandomForestClassifier(random_state=42)
rfc.fit(X_train, y_train)
print('Random Forest Classifier')
rfc_pred = rfc.predict(X_test)
model_report(y_test, rfc_pred)
```

[183] ✓ 0.1s

```
Random Forest Classifier
Confusion Matrix:
 [[  0  13   0]
 [  0 215   7]
 [  0  22  15]]
Classification Report:
              precision    recall  f1-score   support

           0       0.00      0.00      0.00        13
           1       0.86      0.97      0.91       222
           2       0.68      0.41      0.51        37

    accuracy                           0.85       272
   macro avg       0.51      0.46      0.47       272
weighted avg       0.79      0.85      0.81       272

Macro F1 score: 0.4731638418079096
Accuracy: 0.8455882352941176
```

4.3.1 Model performance analysis

A. Confusion matrix:

```
Confusion Matrix:
 [[  0  13   0]
 [  0 215   7]
 [  0  22  15]]
```

- **Class 0 (Low):** 0 correct predictions, 13 misclassified as 1 and 0 misclassified as 2.
- **Class 1 (Medium):** 215 correct predictions, 0 misclassified as 0 and 7 misclassified as 2.
- **Class 2 (High):** 15 correct predictions, 0 misclassified as 0 and 22 misclassified as 1.

B. Classification report:

- **Precision** shows the proportion of correct predictions among all the predictions made for a class.
    - Class 0: precision of 0 means the model never predicted low.
    - Class 1: precision of 0.86 means 86% of the wines predicted as 1 are correct.
    - Class 2: precision of 0.68 means 68% of the wines predicted as 2 are correct.
- **Recall** measures how many wines of a class were correctly identified out of all the wines in that class.
    - Class 0: 0.00 means none of the low wines were found.

    - Class 1: 0.97 means 97% of the medium wines were correctly classified.
    - Class 2: 0.41 means 41% of the high wines were correctly classified.
- **F1-score** is the harmonic mean of precision and recall, used to balance the two.
    - Class 0: 0.00, which is the worst possible result.
    - Class 1: 0.91, which is very good.
    - Class 2: 0.51, which is considered poor to moderate.

C. Accuracy:

The accuracy is 84.6% and the macro F1-score is 0.47.

To summarize:

The model behaves almost the same as SVM, apart from accuracy. The performance metrics show that it classifies class 1 well, struggles with class 2 and does not recognise class 0. All three models are biased towards the majority class, which I address in the hyperparameter tuning section.

4.4 Cross-validation score

Cross-validation is a technique used to evaluate the performance of a machine learning model. It splits the dataset into several parts (folds), trains the model on some folds and tests it on the remaining fold. This is repeated several times and the average of the scores is taken. It is used to understand the performance of different models.

I ran cross-validation for all three models to understand and compare them. I used **StratifiedKFold**, which keeps the same proportion of each class in every fold.

```
Cross Validation Scores
Random Forest Classifier: [0.83823529 0.84191176 0.83455882 0.83455882 0.82287823] Avg:  0.8344285869329282
Gradient Boosting: [0.82352941 0.82720588 0.80882353 0.83823529 0.80442804] Avg:  0.8204444323855004
svm: [0.83088235 0.84191176 0.83823529 0.84558824 0.82656827] Avg:  0.8366371825482961
```

In cross-validation, I observed that the results are very stable across all five folds (80–85%):

- **Random Forest:** 83.4% average accuracy.
- **Gradient Boosting:** 82.1% average accuracy.
- **SVM:** 83.7% average accuracy, the highest.

These results are very close to the test-set accuracy, so the models are not depending on one lucky split. However, they are only slightly above the baseline of 81.8%, which confirms that the models mostly predict the medium class.

4.5 Hyperparameter tuning

In machine learning, hyperparameter tuning is a useful step for improving model performance. Hyperparameters are settings that control the training process of a model. Tuning them lets us choose the configuration that works best for our data, with the aim of getting a better result.

For this project I used two common methods for hyperparameter tuning, RandomizedSearchCV and GridSearchCV. Both help to find the best combination of hyperparameters.

- **A. RandomizedSearchCV:** this method randomly selects combinations of parameters from the ranges I configured and evaluates each one with cross-validation. I chose it because it is computationally cheaper than GridSearch, as it does not try every possible combination.
- **B. GridSearchCV:** this method tries every possible combination of the hyperparameters I configured and evaluates each one with cross-validation. I found this method more time-consuming.

Because the default models ignored the low class, I made two changes for the tuning:

1. I used the **macro F1-score** as the scoring metric instead of accuracy.
2. I added **class_weight='balanced'** as an option, which gives more importance to the rare classes during training.

By tuning the hyperparameters after training, I was able to improve the models' performance on the minority classes.

4.5.1 RandomizedSearchCV for the Random Forest Classifier

```
# use randomized search to find the best parameters
rfc_rs = RandomForestClassifier(random_state=42)

param_dist = {
    'n_estimators': [100, 200, 300],
    'max_depth': [5, 10, 15, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'class_weight': [None, 'balanced']
}

random_search = RandomizedSearchCV(rfc_rs, param_dist, n_iter=30, scoring='f1_macro',
                                   cv=skf, random_state=42, n_jobs=-1)
random_search.fit(X_train, y_train)
print('Random Forest Classifier')
print('Best Parameters:', random_search.best_params_)
print('Best Score:', random_search.best_score_)
```

[186] ✓ 6.6s Python

```
Random Forest Classifier
Best Parameters: {'n_estimators': 100, 'min_samples_split': 5, 'min_samples_leaf': 1, 'max_depth': 5, 'class_weight': 'balanced'}
Best Score: 0.522945658359515
```

From the output, the best parameters are:

- Number of trees: 100
- Minimum samples split: 5
- Minimum samples leaf: 1
- Max depth: 5
- Class weight: balanced

The best cross-validation macro F1-score was 52% (0.523). For comparison, the default models reached a macro F1-score of only 0.47–0.50 on the test set, so balanced class weights clearly help the model with the rare classes.

4.5.2 GridSearchCV for the Random Forest Classifier

```
[229]  4.2s  Python

# GridSearch
rfc_gs = RandomForestClassifier(random_state=42)

param_grid = {
    'n_estimators': [100, 200],
    'max_depth': [5, 10, None],
    'min_samples_leaf': [1, 2, 4],
    'class_weight': [None, 'balanced']
}

grid_search = GridSearchCV(rfc_gs, param_grid, cv=skf, scoring='f1_macro', n_jobs=-1)
grid_search.fit(X_train, y_train)
print('Random Forest Classifier Grid Search')
print('Best Parameters:', grid_search.best_params_)
print('Best Score:', grid_search.best_score_)

rf_gs_pred = grid_search.best_estimator_.predict(X_test)
model_report(y_test, rf_gs_pred)
```

```
Random Forest Classifier Grid Search
Best Parameters: {'class_weight': 'balanced', 'max_depth': 5, 'min_samples_leaf': 2, 'n_estimators': 100}
Best Score: 0.5277776713867215
Confusion Matrix:
 [[  6   7   0]
 [ 12 171  39]
 [  0  10  27]]
Classification Report:
              precision    recall  f1-score   support

           0       0.33      0.46      0.39        13
           1       0.91      0.77      0.83       222
           2       0.41      0.73      0.52        37

    accuracy                           0.75       272
   macro avg       0.55      0.65      0.58       272
weighted avg       0.81      0.75      0.77       272

Macro F1 score: 0.581838320105719
Accuracy: 0.75
```

From the output, the best parameters are:

- Number of trees: 100
- Minimum samples leaf: 2
- Max depth: 5
- Class weight: balanced

The cross-validation macro F1-score became 53% (0.528), slightly higher than with **RandomizedSearchCV**. On the test set the macro F1-score became 58%, compared with 47% for the default Random Forest. The tuned model now correctly identifies:

- **6 of the 13 low wines**, compared with 0 before tuning.
- **27 of the 37 high wines**, compared with 15 before tuning.

The accuracy dropped to **75%,** because some medium wines are now predicted as low or high. Even so, the model is much more useful, because it recognizes all three classes.

Conclusion

To train the model and find a good score, I first loaded the dataset. Then I explored, analyzed and visualized the data. I cleaned it by removing 240 duplicates, normalized the skewed features with a log transformation, treated outliers with clipping, reduced the correlated features with PCA, and standardized the data. After that, I grouped the quality scores into three classes and split the dataset into training and testing sets. I selected three machine learning models and used cross-validation and hyperparameter tuning to improve their performance.

Finally, I analyzed the performance of each model. The default models reached a high accuracy (84–85%), but they did not recognize any low-quality wines, because the dataset is strongly imbalanced. After tuning with balanced class weights and the macro F1-score, the best model for this dataset was the Random Forest Classifier tuned with GridSearchCV, which had the highest cross-validation macro F1-score (0.528). On the test set it achieved an accuracy of 75% and a macro F1-score of 0.58, and, unlike the default models, it recognized all three quality classes: 46% of low, 77% of medium and 73% of high wines. The most important features for predicting quality were alcohol, volatile acidity and sulphates, as shown in the correlation analysis.

Limitations: the scaler and PCA were fitted on the whole dataset before the train/test split, which may cause a small amount of data leakage. The low-quality class is also very small (only 13 wines in the test set), so its results can change with a different split. More data for the rare classes, or oversampling techniques such as SMOTE, could improve the results further.